

UNIVERSIDAD GERARDO BARRIOS
FACULTAD DE CIENCIA Y TECNOLOGÍA
INGENIERÍA EN SISTEMAS Y REDES COMPUTACIONALES



Modulo 4 Desarrollo de aplicaciones con inteligencia artificial

TEMA:

Preparación del proyecto para despliegue, contenedor o simulación

PRESENTADO POR:

Bryan Orlando Giron Argueta
Gerson Usiel Quintanilla Sánchez
Saúl Emmanuel Zuniga Villatoro

Docente: Marco Antonio Arévalo Zambrano

EL SALVADOR, SAN MIGUEL, 1 de AGOSTO DE 2026

1. Descripción breve del servicio preparado

GameVision IA es una aplicación que predice el potencial comercial de videojuegos en Steam, usando un modelo de Machine Learning (Random Forest) entrenado con datos de aproximadamente 58,000 juegos. El servicio preparado esta semana es el backend: una API construida con FastAPI que expone endpoints de predicción, un chatbot (LangChain + Gemini) y autenticación vía Google OAuth (Supabase Auth), conectado a una base de datos Postgres administrada por Supabase.

Esta semana se empaquetó el backend completo en un contenedor Docker reproducible, se desplegó públicamente en Render, y se automatizó el flujo de despliegue para que solo ocurra después de que las pruebas automatizadas (Semana 3) pasen.

2. Ruta elegida

Contenedor Docker, desplegado en la plataforma PaaS Render.

Se eligió un único contenedor para el backend, dejando la base de datos (Supabase) y el frontend (Vercel, pendiente de desplegar) como servicios administrados independientes, fuera de Docker.

Componente	¿Dentro de Docker?	Dónde vive
Backend (FastAPI + IA)	Sí — único contenedor	Render (Web Service, Docker)
Base de datos	No	Supabase (Postgres administrado)
Autenticación	No	Supabase Auth (Google OAuth)
Modelo entrenado (rf_model.pkl)	Se descarga durante el build	GitHub Releases
Frontend	No	Vercel (pendiente de desplegar)

3. Dockerfile

Build multi-etapa: la primera etapa instala dependencias y descarga el modelo; la segunda copia solo lo necesario, corre con un usuario sin privilegios (appuser) y produce una imagen final más liviana.

```
# ---- Etapa 1: construir dependencias y descargar el modelo ----
FROM python:3.11-slim AS builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    libpq-dev \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Entorno virtual aislado, evita instalar como root en /root/.local
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Descargar el modelo entrenado (63MB) desde GitHub Releases
RUN curl -L -o rf_model.pkl https://github.com/sauzuniga/GameVisionIA/releases/download/model-v1.0.0/rf_model.pkl

# ---- Etapa 2: imagen final, mas liviana ----
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 \
    && rm -rf /var/lib/apt/lists/*

ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

# Usuario sin privilegios, en vez de correr la app como root
RUN useradd --create-home --shell /bin/bash appuser

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY --from=builder /app/rf_model.pkl .
COPY . .

RUN chown -R appuser:appuser /app
USER appuser

EXPOSE 8000

CMD ["sh", "-c", "uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

4. .dockerignore

Excluye del contexto de build todo lo que no debe (o no necesita) estar dentro de la imagen de producción:

```
venv/  
__pycache__/  
**/__pycache__/  
*.pyc  
*.pyo  
.pytest_cache/  
.ruff_cache/  
tests/  
.env  
*.db  
*.pkl  
.git  
.gitignore  
requirements-dev.txt  
.vscode/  
.idea/  
.DS_Store  
Thumbs.db
```

Explicación de las exclusiones principales

- venv/, __pycache__/, *.pyc — artefactos de Python que se regeneran, no se necesitan en la imagen
- tests/, requirements-dev.txt — herramientas de desarrollo (pytest, ruff); las pruebas ya corren en GitHub Actions antes del build, no dentro del contenedor
- .env — nunca debe copiarse a la imagen; las variables reales se inyectan en tiempo de ejecución desde Render
- *.pkl — el modelo se descarga dentro del contenedor con curl, no se copia desde la máquina local (evita subir 63MB innecesarios al build)
- .git, .gitignore — no aportan nada al contenedor en ejecución

5. Variables de entorno

Documentadas en backend/.env.example (sin valores reales). En producción se configuran directamente en el panel "Environment" de Render — nunca en el código ni en la imagen Docker.

Variable	Propósito	Dónde se configura
DATABASE_URL	Connection string de Postgres (Session Pooler de Supabase + sslmode=require)	Render → Environment
GEMINI_API_KEY	Clave de la API de Google Gemini para el chatbot	Render → Environment
SUPABASE_JWT_SECRET / JWKS	Verificación de tokens JWT emitidos por Supabase Auth	Render → Environment
ALLOWED_ORIGINS	Dominios permitidos por CORS (URL del frontend)	Render → Environment
PORT	Puerto asignado dinámicamente por Render	Injectada automáticamente por Render

6. Evidencia de construcción, ejecución y despliegue

6.1 Construcción y ejecución local (Docker Desktop)

```
(background on this error at: https://sqa.ionic.me/c/20/csq0)
PS D:\gamevisionIA\backend> docker run -p 8000:8000 --env-file .env gamevision-b
[INFO] Modelo cargado correctamente
INFO:      Started server process [7]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      172.17.0.1:36726 - "GET /health HTTP/1.1" 200 OK
INFO:      172.17.0.1:34200 - "GET /health HTTP/1.1" 200 OK
INFO:      172.17.0.1:42606 - "GET /docs HTTP/1.1" 200 OK
INFO:      172.17.0.1:42606 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:      172.17.0.1:36808 - "POST /api/predict-demo HTTP/1.1" 200 OK

got 3 SIGTERM/SIGINTs, forcefully exiting
PS D:\gamevisionIA\backend> docker ps
CONTAINER ID   IMAGE                                COMMAND                                            CREATED        STATUS
NAMES
cfad728922d4   gamevision-backend                 "sh -c 'uvicorn main..."                      7 minutes ago   Up
000/tcp       loving_grothendieck
PS D:\gamevisionIA\backend> docker stop cfad728922d4
cfad728922d4
```

6.2 Configuración del servicio Docker en Render

Se creó un Web Service de tipo Docker en Render, conectado al repositorio de GitHub. Se configuró backend/Dockerfile como ruta del Dockerfile y backend como contexto de construcción. Las variables sensibles se agregaron desde el apartado Environment.

WEB SERVICE

GameVisionIA

Docker

Free

Upgrade your instance →

Connect ▾

Manual Deploy ▾

Service ID: `srv-d9mg6fm417fc73bbldl0`

sauzuniga / GameVisionIA main

<https://gamevisionia.onrender.com>

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

[Upgrade now](#)

July 31, 2026 at 4:10 PM Live

`84ef912` ci: agregar job de CD para desplegar automaticamente en Render

All logs ▾

Jul 31, 4:09 PM - 4:12 PM ▾

CST

Jul 31

04:11:50 PM ==>

04:11:50 PM ==> //////////////////////////////////////

04:11:55 PM [6d5kq] INFO: 10.213.26.204:60784 - "GET /health HTTP/1.1" 200 OK

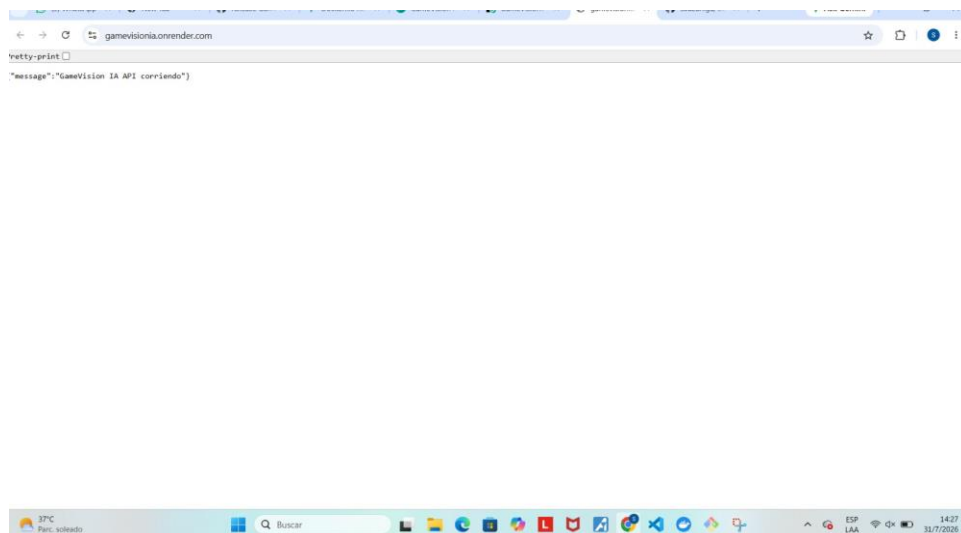
04:11:59 PM [6d5kq] INFO: 10.213.26.204:60796 - "GET /health HTTP/1.1" 200 OK

04:12:00 PM [6d5kq] INFO: 10.213.26.204:60804 - "GET /health HTTP/1.1" 200 OK

04:12:05 PM [6d5kq] INFO: 10.213.26.204:43456 - "GET /health HTTP/1.1" 200 OK

04:12:10 PM [6d5kq] INFO: 10.213.26.204:43472 - "GET /health HTTP/1.1" 200 OK

6.3 Servicio desplegado y accesible públicamente



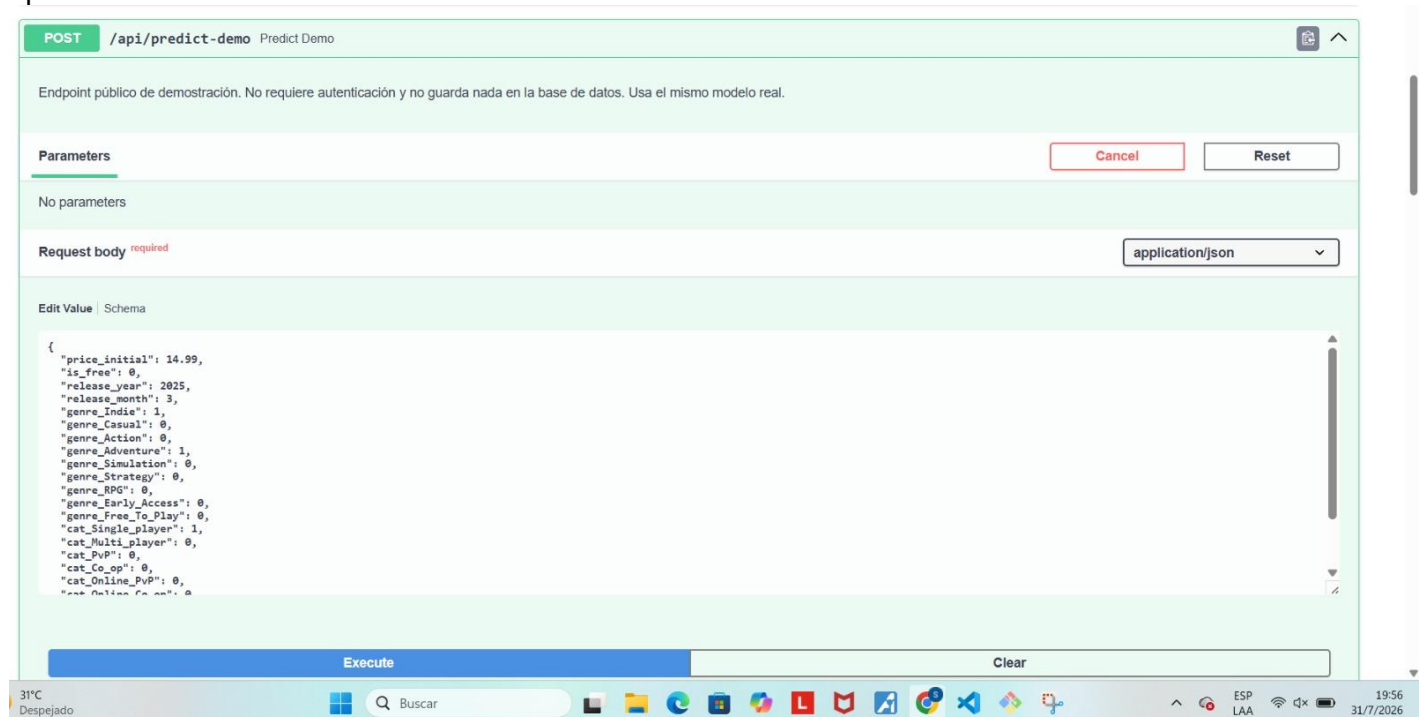
7. Prueba del endpoint /health

Se prueba el endpoint/health desde la url de render



8. Prueba del endpoint principal (/api/predict-demo)

Se realiza una predicción desde el endpoint principal, es decir se prueba la función principal de la aplicación.



Server response

CodeDetails

200

Response body

```
{
  "result": "Bajo",
  "confidence": 0.4645,
  "predicted_class": 0,
  "potential_level": "Bajo",
  "model_version": "v1.0.0",
  "warnings": [],
  "request_id": "39df93a8",
  "demo": true,
  "note": "Endpoint de demostraci\u00f3n. No requiere autenticaci\u00f3n y no guarda historial."
}
```

 Download

Response headers

```
access-control-allow-credentials: true
alt-svc: h3=":443"; ma=86400
cf-cache-status: DYNAMIC
cf-ray: a23f5cc37fee146c-ATL
content-encoding: br
content-length: 183
content-type: application/json
date: Fri, 31 Jul 2026 20:30:41 GMT
rndr-id: 70075e49-ea96-4755
server: cloudflare
vary: Accept-Encoding
x-render-origin-server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

9. Errores, intentos, bloqueos y correcciones realizadas

9.1 Ruta del Dockerfile no encontrada en Render

Configurar únicamente "Root Directory: backend" no fue suficiente para que Render localizara el Dockerfile (proyecto monorepo con backend/ y frontend/).

```
Jul 31
02:19:14 PM ==> Cloning from https://github.com/sauzuniga/GameVisionIA
02:19:20 PM ==> Checking out commit 9d5300941b60f45edba8280fa2a55658462b0403 in branch main
02:19:22 PM #1 [internal] load build definition from Dockerfile
02:19:22 PM #1 transferring dockerfile: 2B done
02:19:22 PM #1 DONE 0.0s
02:19:22 PM error: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
```

Corrección: se configuraron explícitamente "Dockerfile Path: backend/Dockerfile" y "Docker Build Context Directory: backend" en Settings del servicio.

9.2 Conexión IPv6 inalcanzable dentro de la red de Docker

La conexión directa a Supabase (db.xxxxx.supabase.co) resuelve solo a IPv6; la red virtual de Docker Desktop no tenía ruta de salida IPv6, produciendo "Network is unreachable". Corrección: se cambió a la URL del Session Pooler de Supabase (IPv4) con sslmode=require, el patrón recomendado oficialmente por Supabase para servidores de larga duración.

9.3 Alcance de working-directory en workflow multi-job

Al extender el pipeline de CI a CI/CD, el nuevo job de despliegue falló porque heredaba una configuración de directorio de trabajo pensada solo para el job de pruebas (que sí descarga el repositorio; el job de despliegue no lo necesita).

```
CD - Solicitar despliegue en Render
failed now in 4s
Search logs Explain error

> Set up job 0s
v Activar Deploy Hook de Render 0s
1 ▶ Run if [ -z "$RENDER_DEPLOY_HOOK_URL" ]; then
17 Error: An error occurred trying to start process '/usr/bin/bash' with working directory '/home/runner/work/GameVisionIA/GameVisionIA/backend'. No such file or directory
> Complete job 0s
```

Corrección: se movió la configuración de directorio de trabajo para que aplique únicamente al job de pruebas, no a todo el workflow.

9.4 Buenas prácticas de seguridad en el Dockerfile

Se incorporaron 3 mejoras antes del despliegue final: entorno virtual aislado (evita instalar como root), usuario sin privilegios (appuser) en vez de root, y variables PYTHONUNBUFFERED / PYTHONDONTWRITEBYTECODE para logs en tiempo real.

10. Plan de infraestructura mínima

Componente	Servicio	Plan	Función
------------	----------	------	---------

Backend	Render	Free (Docker)	Único contenedor: API + IA
Base de datos	Supabase	Free	Predicciones, sesiones, historial
Autenticación	Supabase Auth	Free	Google OAuth, JWT (ES256)
Modelo entrenado	GitHub Releases	Gratuito	Descarga durante el build
CI/CD	GitHub Actions	Gratuito	Pruebas + despliegue automático

Este conjunto es el mínimo viable: un solo contenedor proporcional al tráfico real del proyecto, servicios administrados en vez de infraestructura propia, y almacenamiento del modelo sin costo adicional vía GitHub Releases.

11. Estimación de costos iniciales

Costo actual: \$0/mes (todos los componentes en plan gratuito).

Supuesto: tráfico bajo, propio de evaluación académica — no producción con usuarios concurrentes reales.

Servicio	Plan pago mínimo	Costo aprox.	Por qué se necesitaría
Render	Starter	US\$7/mes	Elimina el "spin down" tras inactividad
Supabase	Pro	US\$25/mes	Más almacenamiento, backups diarios
Vercel	Pro	US\$20/mes	Uso comercial, mayor ancho de banda

Total estimado en producción real: ~US\$52/mes. Supuestos: tráfico bajo-medio, sin dominio propio, uso del chatbot dentro de la cuota gratuita de Gemini.

12. Riesgos técnicos pendientes

Riesgo	Probabilidad	Impacto	Estado
RLS no activado en Supabase	Media	Alto	Pendiente — prioridad próxima semana
OAuth de Google en modo Testing	Alta	Medio	Pendiente — publicar antes de demo final
Chatbot depende 100% de Gemini (sin fallback)	Media	Alto	Pendiente — manejo de error más robusto
Sin Network Restrictions en Supabase	Baja	Medio	Pendiente — capa extra de seguridad
Spin-down del plan gratuito de Render (~50s de retraso)	Alta	Bajo	Aceptado — limitación conocida del tier gratuito
Un solo contenedor, sin redundancia	Baja	Medio	Aceptado — apropiado para el tamaño actual
Sin migraciones formales (Alembic)	Media	Medio	Pendiente — no bloqueante por ahora

Proyecto expuesto con render: <https://gamevisionia.onrender.com/docs>

Repositorio: <https://github.com/sauzuniga/GameVisionIA.git>